

Advances in Biomedical Informatics



Applications of Computational Intelligence in Multi-Disciplinary Research

Series Editor: **Valentina Emilia Balas**

Edited by

**Ahmed A. Elngar, Rajdeep Chowdhury,
Mohamed Elhoseny and Valentina Emilia Balas**



Advances in Biomedical Informatics

Applications of Computational Intelligence in Multi-Disciplinary Research

Edited by

Ahmed A. Elngar

Faculty of Computers and Artificial Intelligence, Beni-Suef University, Beni-Suef City, Egypt
College of Computer Information Technology, American University in the Emirates,
United Arab Emirates

Rajdeep Chowdhury

Department of Computer Application, JIS College of Engineering, Kalyani, West Bengal, India

Mohamed Elhoseny

College of Computing and Informatics, University of Sharjah, United Arab Emirates
Faculty of Computers and Information, Mansoura University, Egypt

Valentina Emilia Balas

Department of Automatics and Applied Software, Faculty of Engineering,
"Aurel Vlaicu" University of Arad, Arad, Romania

Series Editor

Valentina Emilia Balas

Department of Automatics and Applied Software, Faculty of Engineering,
"Aurel Vlaicu" University of Arad, Arad, Romania



ACADEMIC PRESS

An imprint of Elsevier

Academic Press is an imprint of Elsevier
125 London Wall, London EC2Y 5AS, United Kingdom
525 B Street, Suite 1650, San Diego, CA 92101, United States
50 Hampshire Street, 5th Floor, Cambridge, MA 02139, United States
The Boulevard, Langford Lane, Kidlington, Oxford OX5 1GB, United Kingdom

Copyright © 2022 Elsevier Inc. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or any information storage and retrieval system, without permission in writing from the publisher. Details on how to seek permission, further information about the Publisher's permissions policies and our arrangements with organizations such as the Copyright Clearance Center and the Copyright Licensing Agency, can be found at our website: www.elsevier.com/permissions.

This book and the individual contributions contained in it are protected under copyright by the Publisher (other than as may be noted herein).

Notices

Knowledge and best practice in this field are constantly changing. As new research and experience broaden our understanding, changes in research methods, professional practices, or medical treatment may become necessary.

Practitioners and researchers must always rely on their own experience and knowledge in evaluating and using any information, methods, compounds, or experiments described herein. In using such information or methods they should be mindful of their own safety and the safety of others, including parties for whom they have a professional responsibility.

To the fullest extent of the law, neither the Publisher nor the authors, contributors, or editors, assume any liability for any injury and/or damage to persons or property as a matter of products liability, negligence or otherwise, or from any use or operation of any methods, products, instructions, or ideas contained in the material herein.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

Library of Congress Cataloging-in-Publication Data

A catalog record for this book is available from the Library of Congress

ISBN: 978-0-12-823978-0

For Information on all Academic Press publications
visit our website at <https://www.elsevier.com/books-and-journals>

Publisher: Mara Conner
Acquisitions Editor: Chris Katsaropoulos
Editorial Project Manager: Isabella C. Silva
Production Project Manager: Selvaraj Raviraj
Cover Designer: Greg Harris

Typeset by MPS Limited, Chennai, India



Contents

List of contributors

ix

1. Iris feature extraction using three-level Haar wavelet transform and modified local binary pattern	1
<i>Prajoy Podder, M. Rubaiyat Hossain Mondal and Joarder Kamruzzaman</i>	
Abbreviations	1
1.1 Introduction	1
1.2 Related works	3
1.3 Iris localization	4
1.4 Iris normalization	6
1.5 The proposed feature extraction scheme	7
1.6 Matching results	11
1.7 Performance evaluation	11
1.8 Conclusion	13
References	14
 2. A novel crypt-intelligent cryptosystem	 17
<i>Pratyusa Mukherjee and Chittaranjan Pradhan</i>	
2.1 Introduction	17
2.2 Related work	18
2.2.1 Machine learning contributions in cryptology	18
2.2.2 Genetic algorithm contributions in cryptology	20
2.2.3 Neural network contributions in cryptology	21
2.2.4 Background of DNA cryptography	23
2.3 Proposed methodology	23
2.3.1 Proposed encryption scheme	24
2.3.2 Proposed decryption scheme	25
2.4 Discussion	25
2.5 Conclusion and future work	26
References	26

3. Behavioral malware detection and classification using deep learning approaches	29
<i>T. Poongodi, T. Lucia Agnes Beena, D. Sumathi and P. Suresh</i>	
3.1 Introduction	29
3.1.1 Digital forensics—malware detection	30
3.1.2 Malware evolution and its taxonomy	32
3.1.3 Machine learning techniques for malware analysis	32
3.1.4 Behavioral analysis of malware detection	33
3.2 Deep learning strategies for malware detection	35
3.2.1 Feature extraction and data representation	35
3.2.2 Static Analysis	36
3.2.3 Dynamic analysis	38
3.2.4 Hybrid analysis	38
3.2.5 Image processing techniques	38
3.3 Architecture of CNNs for malware detection	41
3.3.1 Preprocessing	41
3.3.2 Classification using CNNs	41
3.3.3 Evaluation	42
3.4 Comparative analysis of CNN approaches	42
3.5 Challenges and future research directions	43
3.6 Conclusion	43
References	43
 4. Optimization techniques and computational intelligence with emerging trends in cloud computing and Internet of Things	 47
<i>Jayesh S Vasudeva, Sakshi Bhargava and Deepak Kumar Sharma</i>	
4.1 Introduction	47

4.1.1	Introduction to optimization	48	6.2	Supervisory control and data acquisition systems	84
4.1.2	Introduction to cloud computing with emphasis on fog/edge computing	48	6.3	The very small aperture terminal networks	87
4.2	Optimization techniques	49	6.3.1	The satellite communication systems	87
4.2.1	An optimization problem	49	6.3.2	Architecture very small aperture terminal networks	89
4.2.2	Solution to the optimization problem	52	6.3.3	Connectivity	92
4.3	Understanding fog/edge computing	54	6.3.4	Multiple access	93
4.3.1	What is fog?	54	6.4	Algorithm for estimating the satellite bandwidth	95
4.3.2	Prelude to our framework	54	6.4.1	Determining the bandwidth required for data transmission	95
4.3.3	Our goal	55	6.4.2	Case study	98
4.3.4	Framework for fog computing	55	6.4.3	Overview of some recent algorithms in detail	99
4.4	Optimizing fog resources	57	6.4.4	Validation of bandwidth calculations	104
4.4.1	Defining optimization problem for fog layer resources	57	6.5	Challenges and future work	106
4.4.2	Optimization techniques used	58	6.6	Conclusions	107
4.5	Case studies	60	References	107	
4.5.1	Case study I: floorplan optimization	60			
4.5.2	Case study II: Gondwana—optimization of drinking water distribution system	63			
4.6	Scope of advancements and future research	63			
4.7	Conclusion	64			
References		65			
5.	Bluetooth security architecture cryptography based on genetic codons	67	7.	Using artificial intelligence search in solving the camera placement problem	109
	<i>Asif Ikbal Mondal, Bijoy Kumar Mandal, Debnath Bhattacharyya and Tai-Hoon Kim</i>			<i>Altahir A. Altahir, Vijanth S. Asirvadam, Nor Hisham B. Hamid and Patrick Sebastian</i>	
5.1	Introduction	67		Nomenclature	109
5.1.1	Bluetooth	67	7.1	Introduction	109
5.1.2	Bluetooth security architecture	67	7.1.1	The roles of visual surveillance systems	110
5.2	Survey of literature	69	7.1.2	The camera placement problem from an artificial intelligence perspective	111
5.3	Plaintext-to-ciphertext conversion process	71	7.1.3	Chapter description	112
5.3.1	Basic workflow	71	7.2	Background	112
5.3.2	Algorithm	72	7.3	Modeling the visual sensors	113
5.3.3	Analysis and discussion	78	7.3.1	The sensor space modeling	114
5.4	Conclusion	79	7.3.2	The camera coverage modeling	114
5.5	Future work	79	7.3.3	The analysis of camera visibility	115
References		80	7.4	Solving the camera placement problem using artificial intelligence search	116
6.	Estimation of the satellite bandwidth required for the transmission of information in supervisory control and data acquisition systems	83	7.4.1	Generate and test algorithm	117
	<i>Marius Popescu and Antoanela Naaji</i>		7.4.2	Uninformed search	118
Abbreviations		83	7.4.3	Hill climbing strategy	119
6.1	Introduction	84	7.5	Further discussion	121
			7.5.1	The efficiency of the algorithms	122
			7.5.2	The performance of the algorithms	123
			7.6	Conclusion	123
			References	124	

8. Nanotechnology and applications	129	9.10 Nanotechnology in pesticides and fertilizers	151
<i>Kanika Dulta, Amanpreet Kaur Virk, Parveen Chauhan, Paras Bohara and Pankaj Kumar Chauhan</i>		9.11 Control of plant pests	152
8.1 Introduction	129	9.12 Concluding remarks	152
8.2 Nanoscience and nanotechnology	130	Consent for publication	152
8.3 Computational nanotechnology	130	Conflict of interest	153
8.3.1 Molecular modeling	131	References	153
8.3.2 Nanodevice simulation	133		
8.3.3 Nanoinformatics	133		
8.3.4 High-performance computing	135		
8.3.5 Computational intelligence	135		
8.4 Applications of computational nanotechnology	137	10. A methodology for designing knowledge-based systems and applications	159
8.4.1 Nanotube-based sensors and actuators	138	<i>Hien D. Nguyen, Nhon V. Do and Vuong T. Pham</i>	
8.4.2 Nanoinformatics for drugs	138	10.1 Introduction	159
8.4.3 Molecular docking	138	10.2 Related work	160
8.4.4 Nanotoxicology	138	10.3 Design the knowledge-based system	160
8.4.5 Other applications	139	10.3.1 The architecture of a knowledge-based system	160
8.5 Conclusion	139	10.3.2 The process for designing the knowledge-based system	162
References	139	10.4 Knowledge base and inference engine of a knowledge-based system	163
		10.4.1 Design the knowledge base	163
9. Advances of nanotechnology in plant development and crop protection	143	10.4.2 Design the Inference engine	164
<i>Rokeya Akter, Md. Habibur Rahman, Md. Arifur Rahman Chowdhury, Manirujjaman Manirujjaman and Shimaa E. Elshenawy</i>		10.5 Applications	168
9.1 Introduction	143	10.5.1 Design an intelligent problem solver for solving solid geometry at high school	168
9.2 Agriculture's nanofarming: a modern frontier	143	10.6 Conclusion and Future work	183
9.3 Synthesis of green nanoparticles and its sources	144	References	184
9.4 Good distribution possibilities allowed by nanoparticles: a modern sustainable agriculture portal	145		
9.5 Nanofertilizers: a good food supply for crops	146		
9.6 Germination, field production, and efficiency enhancement of seed nanomaterials	148	11. IoT in healthcare ecosystem	187
9.7 Plant sensory systems and responses to radical climate change influences nanomaterials	149	<i>Poonam Gupta and Indhra Om Prabha M</i>	
9.8 Nanosensors and nanomaterials: perturbation detection and control	150	11.1 Introduction	187
9.9 Pesticide-based plant safety nanomaterials	150	11.2 Applications of Internet of Things in healthcare	188
		11.2.1 Patient-centric IoT	188
		11.2.2 Hospital-centric IoT applications	189
		11.2.3 IoT benefitting health insurance companies	190
		11.2.4 Pharmaceutical governance	190
		11.3 Implementation methodologies	190
		11.3.1 Fog computing	191
		11.3.2 Edge computing	192
		11.4 Implementation models	193
		11.4.1 Heart disease prediction	193
		11.4.2 Healthcare IoT-based affective state mining using deep convolutional neural networks	194

11.5 Challenges in healthcare IoT	195		
11.5.1 Technology-oriented challenges	195	11.7.3 Quarantined patient care	199
11.5.2 Adapting to remote healthcare and telehealth	195	11.7.4 Public surveillance	200
11.5.3 Data security	195	11.7.5 Safeguarding hygiene	200
11.6 Security issues and defense mechanisms and IoT	196	11.7.6 IoT and robotics	200
11.6.1 Security requirements in healthcare IoT	196	11.7.7 Smart disinfection and sanitation tunnel	201
11.6.2 Attacks on IoT devices	196	11.7.8 Smart masks and smart medical equipment	201
11.6.3 Defensive mechanism	197	11.8 Future of IoT in healthcare	202
11.7 Covid 19—how IoT rose to the global pandemic	198	11.8.1 IoT and 5G	202
11.7.1 About Covid 19	199	11.8.2 IoT and artificial intelligence	202
11.7.2 Decoding the outbreak and identifying patient zero	199	11.9 Conclusion	203
		References	203
		Index	205

A methodology for designing knowledge-based systems and applications

Hien D. Nguyen^{1,2}, Nhon V. Do³ and Vuong T. Pham⁴

¹University of Information Technology, Ho Chi Minh City, Vietnam, ²Vietnam National University, Ho Chi Minh City, Vietnam, ³Hong Bang International University, Ho Chi Minh City, Vietnam, ⁴Sai Gon University, Ho Chi Minh City, Vietnam

10.1 Introduction

Intellectual is the knowledge and the understanding of humans in different areas of expertise or certain knowledge, such as in general mathematics, or specialty subjects like plane geometry, solid geometry, and analytic geometry; knowledge about general physics or in a specialty subject such as alternating current/direct current and mechanics; and knowledge about sicknesses and diseases in the line of medicine. Knowledge often includes abstract compositions together with diversity in connections and complicated relations between the components [1,2]. The components of knowledge that are commonly seen are the concepts, the relations, the operators, the functions, the facts, and the rules; the conceptual knowledge component and their relations between them are the foundation of intellectual [3,4]. Knowledge also includes experiences and regulations that support the process of reasoning and solving problems relating to the understanding of a certain subject or an area of the subject [5,6].

Knowledge about solid geometry in high schools [7] gives us a detailed example of knowledge with complicated components. First, there are three basic concepts that are the base foundation: point, straight line, and plane. The concept theory becomes the base foundation to identify or construct different objects of geometry including rays, straight segments, angles, triangles and specialized triangles, quadrilaterals and specialized quadrilaterals, circumference and arc, pyramids, cylinders, cubes, and spheres. Between the concepts, there is one special relationship called “is a,” also known as a hierarchy onset of concepts. Besides the hierarchy of concepts, there are other concept relations including equal relation, comparative relation on values, parallel relation, perpendicular relation, and similar relation. Almost all of the relations are binary relations, and each relation has different functions/characteristics such as symmetry, asymmetry, and transitive relation. In solid geometry, there are functional knowledge components including functions such as the projection (perpendicular) of a point on a line or a plane; the intersection of two lines and that of two planes; the distance between a point and a line or a plane; and the distance between two parallel lines and that between two parallel planes. There are operators for vectors such as the addition of two vectors, the scalar product of two vectors, or multiplication of a vector by a number. Based on the above knowledge components of solid geometry, there are many rules (theorems, properties, formulas, etc.), such as theorems based on the elements of a triangle or a quadrilateral, Pythagorean theorem, the theorem of equal triangles, and the theorem of three perpendicular lines.

A knowledge-based system (KBS) is a system that uses techniques of artificial intelligence in problem-solving processes to support human action, decision-making, and learning [1,5,6]. This system helps us in managing knowledge of different knowledge domains on the computer and helps in solving knowledge-related problems; more advanced is the system with the capability of reasoning or inference dealing with complex and abstract problems [3,8]. The system usually has an organization that stores knowledge for solving application requirements or problems; this knowledge organization is called the knowledge base (KB) of the system [1,2,9].

The KBS can be classified based on aspects with certain specific standards such as openness, knowledge representation, application objectives, and architecture, as well as operation of the system, or based on the application domain [1,9,10]. A KBS has a KB built with some initial “field knowledge,” and it is only able to use that knowledge during its operation or during its lifetime. An open KBS is capable of adding knowledge during the operation, often with a knowledge discovery

module based on machine learning methods. The combined KBSs have features that result from a combination of closed and open systems, a combination of the KB and database, a combination of one KB and another, etc.

This chapter is designed to help users develop an appreciation for KBS and its architecture. The proposed method can give a broad variety of knowledge-based techniques for supporting in making decisions and planning. This chapter presents a KB design process and an inference engine (IE) to build intelligent systems for use in education and engineering. For illustrating the effectiveness of this design method, it is used to build an intelligent support system that solves high-school-level solid geometry [11] and a consultancy system for housing architecture based on the regulations of architecture and construction [12].

The next section presents some related works for building intelligent systems based on a KB. Section 10.3 describes the general process to construct a KBS. Section 10.4 proposes the detailed process for designing the components of the KB and IE in KBS. There are some basic knowledge manipulations mentioned in those processes. Section 10.5 presents the application of the proposed method to build two supporting systems in the domains of learning and building: an intelligent problem solver applied in solid geometry at high school and a consultancy system for designing housing architecture based on the regulations of architecture. The last section concludes some main results of this chapter and presents some future works to improve those results.

10.2 Related work

At present, intelligent systems, which are applied in practical fields, require that the design of a KB fully represents knowledge in the field as well as solve common problems and problems that require the intensive application of knowledge [3,13]. Therefore the method of designing and organizing the KB is a very essential need for the process of building intelligent systems [14]. Besides, to solve knowledge problems, the system also needs to have a strong IE to be able to grasp and solve the problems [15]. The configuration of a KBS includes a set of concepts (objects) in the knowledge domain and their properties, a set of relations between the domain concepts, and a task specification describing the demands that a created configuration has to accomplish [16]. There are many studies to build KBs for intelligent systems. Some methods that have been used for KB configuration are as follows: rule-based systems [17], concept hierarchies [18,19], structure and constraint-based approaches [20], and model-based systems [15,21,22]. However, those methods have still not been applied for practical systems.

The designing of a KBS involves three stages: (1) determining the design task, (2) constructing and initializing the design process model, and (3) executing the design process model [9,23]. This method can create an intelligent system with a certain KB. Nevertheless, this design method is too general; it is not really effective to construct a system applied in a practical knowledge domain.

Yoshizumi et al. [10] have studied some methods to construct dynamic KBSs. Human activities have some dynamic aspects corresponding to the working environment. Each method is used to design a type of human activity. The knowledge in this study has an element called “tacit knowledge.” Tacit knowledge, also called implicit knowledge, is the kind of knowledge that is expressed when solving problems of the knowledge domain [24]. It is also a kind of experiential knowledge (heuristic). However, those results did not give a general process to build a usable KBS.

Bobadilla et al. [25] have proposed some methods for designing a kind of intelligent system, known as the recommendation system. Memory-based methods [26,27] use similarity metrics to evaluate the distance between two objects and rate for k-nearest neighbors (kNN) [28,29]. Model-based methods use the information to create a model to generate the results of the system [22,30,31]. However, those results did not present a full process to build a complete intelligent system, from collecting data or knowledge to constructing and testing a practical program, and especially, they did not present how to organize the KB and IE of those systems.

The method for designing KBS has to include the solutions to organize its KB and design its IE. Those constructed components need to satisfy some criteria for application [13,32], such as the KB’s completeness and consistency and the simulation of the human method by the inference process for problem-solving [15,33]. Besides, the method also ensures the usability of the created system. It can solve some practical problems of the applied scope.

10.3 Design the knowledge-based system

10.3.1 The architecture of a knowledge-based system

Architecturally, according to [32,33], the KBS can include components such as the KB, the IE, the interface, explanation module, knowledge manager module, and working memory; the system serves two types of users, who mainly are

normal-type users or the knowledge engineer. Fig. 10.1 is a common architectural diagram of a KB, expert system, or intelligent problem-solving system.

In the system's components, according to the above diagram, the KB and the IE are the two core components. The KB contains the knowledge of the applied knowledge domain as the foundation for computational inference to solve problems or requirements posed for the system. The knowledge needs to be represented and stored in a readable, appropriate form and used by an IE to perform automatic inference in order to find solutions to problems, especially in general forms. The KB stores many knowledge components such as concepts and entities, relations, functions, operators, and facts and inference rules. The design of the KB requires the designer to have a firm grasp of the methods of knowledge representation to apply, modify, and research and develop those representational methods.

The IE will implement problem-solving or inference strategies and find answers to the questions, based on the use of knowledge in the KB. The IE plays the role of demonstrating the system's ability to "think," simulating the human's ability to think and solve problems. Technically, each step of the IE must know to find the appropriate rules and apply the rules to generate new facts (in the forward-chaining strategy) or new goals (in the backward-chaining strategy).

There must be relative independence between the KB and the IE, meaning they are not bound organically. *With the adjustment of either or both components, the system can still operate normally.* The separation of knowledge from control mechanisms makes it easier to add new knowledge in the process of developing a program. This is a similarity of the IE in a KB and the human brain (processing control), which is consistent and does not change even though the individual's behavior changes according to the newly acquired experience and knowledge. Assuming an expert uses traditional programs to assist with day-to-day tasks, the program behavior change requires them to know how to install the program. In other words, the expert must be a professional programmer. This limitation is solved when experts approach using the KBS. In the KBSs, knowledge is expressed explicitly, not in a hidden form as in the traditional programs. Therefore it is possible to change the KB, and the IEs will work on the updated knowledge to fulfill the new request of the experts [13].

The reasons why ensuring the separation between the KB and the IE is an important criterion:

- The separation of the knowledge of problem-solving and the IE will help the representation of the knowledge to operate in a more natural way, closer to human conception.
- The designers of intelligent systems will focus on capturing and organizing the KB rather than going into details for the installation on a computer.

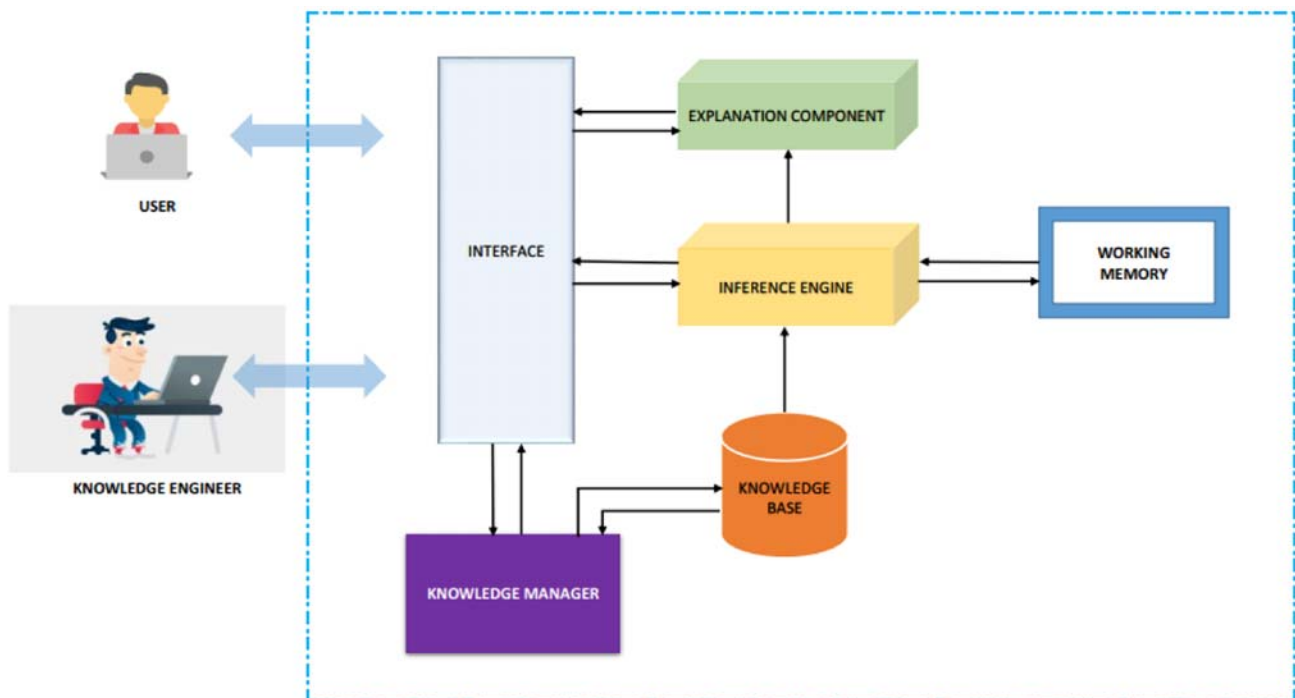


FIGURE 10.1 Architectural diagram of a knowledge-based system.

- This separation will enhance the modularization of the KB, IE, and the knowledge-updating system. The addition or removal of a part of the knowledge will not have marginal effects on the other components of the system.
- The same control and communication strategy is allowed to be used for many different systems.
- The separation of the KB and IE also makes it possible to test a variety of control strategies on the same KB.

The basic problem-solving operation of the system is as follows: the problem is entered into the system through the interface and the IE analyzes the problem and records the facts (hypothesis and goals) into the working memory. Following that, the IE performs the inference strategies for finding the answer or solution to the problem. Throughout the reasoning process, the IE looks for rules in the KB to apply to find the solution.

For a system that solves plane geometry problems, for example, the system has a KB with the concepts of plane geometry such as relations, properties, theorems, and formulas. The user will enter the problem into the system including hypothesis and conclusions (or requirements). In order to find a solution to the problem, the IE performs an inference process to look for theorems, formulas, etc. to apply to the search for the answer. Then the solution of the problem is exported to the user.

Learning-aid applications also have a need for instruction systems that solve problems, such as a problem-solving instruction system for plane geometry. In this system, in addition to the KB and the IE, there is also a module to guide how to solve exercises. These modules simulate the activities of the teacher when teaching the students to solve problems in system operations based on constructive algorithms, search algorithms, and solution optimization algorithm. The act of teaching students how to solve math problems demonstrates intelligent characteristics such as pedagogy and interactivity, which helps to create an interest in learning and to stimulate the creative thinking process of students. The application of instructions for solving plane geometry needs to meet these basic requirements, given as follows:

- The KB of the system stores all the knowledge and information of the knowledge domain about plane geometry including an intellectual repository and exercises that are categorized and linked with the KB [34].
- There are features to support solving geometry exercises to ensure a number of requirements such as pedagogical psychology, interactivity, scientific properties, usage of the suitable KB with the knowledge of high school plane geometry, and a friendly interface [13,34].
- There is a search function to look up the knowledge and exercises.

10.3.2 The process for designing the knowledge-based system

Based on the architecture of the KBS, the process of designing the KBS includes the following stages and technical problems:

- **Stage 1:** *Collect the knowledge and practical problems.*
- Determine the scope of the knowledge domain with the demanded requirements.
- Determine the source of the collected knowledge domain. The collection sources are regular documents and books and the knowledge of experts. For selecting the experts, it is necessary to consider the selection criteria in accordance with the actual conditions. The collected knowledge usually includes concepts, objects, relationships, facts and inference rules, etc. Knowledge can be preliminarily classified as the foundation for the design of a KB of the system.
- Collect the practical problems, requirements, and questions. From that, the problems will be classified to design an IE.
- **Stage 2:** *Design the KB.*

This stage uses the collecting results of Stage 1 to build the KB of the system. It has some steps:

- Represent the collected knowledge that needs to be organized in the KB. This requires the application of knowledge models and studying methods of knowledge representation.
- Determine the organization of KB based on the built knowledge model.
- Design the basic knowledge manipulations of the KB and some basic tasks of the IE.
- **Stage 3:** *Design the IE.*

This stage is implemented based on the KB built in Stage 2 and the collected problems in Stage 1. The steps for designing an IE are as follows:

- Classify collected problems to determine some general kinds of problems. The models for problems are proposed based on the kinds of practical problems.

- Select the inference strategies corresponding to the kind of problems, and design algorithms to the corresponding kind. This requires the application of deduction methods and studying to improve those methods suitably.
- Test, evaluate, and increase the inference ability for finding the solutions effectively.
- **Stage 4:** *Design the user interface of the KBS.*

There are two kinds of users for a KBS: knowledge engineers and normal users.

- Design the user interface for normal users to input the problems and requirements. It has a specification language similar to natural language for commuting to achieve human interactions easily.
- Design the user interface for the knowledge engineer managing the KB by using the module of knowledge management.
- Besides the input interface, it also has an interface showing the proof of problems. The solutions of the proofs are natural, similar to human expression.

10.4 Knowledge base and inference engine of a knowledge-based system

10.4.1 Design the knowledge base

Based on the collection of knowledge and practical problems which have been solved in the KBS, the collected knowledge is represented to be stored in the KB of the system. This work requires intellectual engineers to use the methods of knowledge representation smoothly, flexibly, and creatively. Using the knowledge model, the engineers organize the KB of the system. In addition to the organized KB, some basic knowledge manipulations also have to be designed, such as updating the knowledge and unifying facts, as the foundation for designing the IE in the next step.

10.4.1.1 Organize the knowledge base

Knowledge representation is the most important work for designing a KB. In this work, the knowledge model must be studied to conform to the collected knowledge. The representing of knowledge accurately and fully helps the KBS to be equipped with a complete KB so that the system can understand the specified intellectual and use them to solve practical problems in the knowledge domain [1,2]. Knowledge representation is the building of a knowledge model for the knowledge domain. This model establishes how the knowledge stored on the computer is used to serve the knowledge process as well as design the IE for problem-solving [3,13,15] (Fig. 10.2).

After the collection, the knowledge is described in the form of natural language. Thus some techniques have to be used to specify this knowledge, such as using visual diagrams (graphs, images), data structures (basic and advanced, abstract), mathematical structures, and specification language [13,33]. The methods for knowledge representation are constructed through the following steps [13]:

- **Step 1:** Analyze the collected knowledge to determine the required components of the knowledge. In this step, the knowledge can be classified into those components. This makes the selection of methods for knowledge representation easier.
- **Step 2:** Determine the relations between components in Step 1.
- **Step 3:** Select the model of knowledge conforming to the knowledge domain. The model can be improved from another model. The basic components of a knowledge model are concepts, relations between concepts, and inference rules.
- **Step 4:** Based on the above steps, construct a complete knowledge model for all collected knowledge by assembling, integrating, and combining the knowledge components.

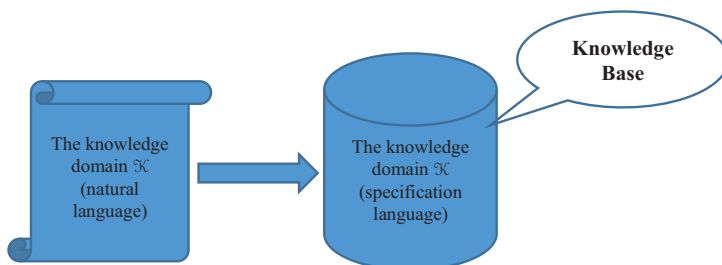


FIGURE 10.2 Knowledge representation.

After the knowledge representation, the knowledge must be stored on the computer for it to become a KB of the KBS. This KB has to satisfy some criteria [13,15]:

- **Completeness:** The knowledge is represented and organized on the computer sufficiently.
- **Consistency:** The organized KB ensures consistency. It does not conflict with other internal knowledge of the system.

There are some techniques to organize the KB based on the knowledge model:

- Method 1: Store the KB by using the system of structured text files based on the keywords and some basic grammar rules.
- Method 2: Store the KB by using relational databases (MySQL, SQL Server, MS Access, etc.)
- Method 3: Use a specification language, such as XML, RDF, or OWL, and some supporting tools for ontology.

10.4.1.2 Basic knowledge manipulations

This section presents some basic knowledge manipulations for the organized KB. These manipulations are the foundation to designing the IE of the KBS. Some basic manipulations are usually used in real-world KBSs:

10.4.1.2.1 Updating the knowledge base

In the real world, the knowledge can be changed and updated, so the designing of KB has a function for updating the stored KB. For updating the knowledge, the KB determines the conformation to the new knowledge, whether the updating affects the working of the system or not, and contradiction with the current knowledge.

10.4.1.2.2 Checking the consistency of the knowledge base

The knowledge must be consistent with other internal knowledge. It means the components of knowledge must conform to the structure of each component and the relations between them. Besides, the KB ensures that the knowledge and the IE do not create conflicting knowledge. Especially when updating the new knowledge of concepts, relations, facts, or rules, the consistency of the KB needs to be checked when designing it.

10.4.1.2.3 Unification of facts

Facts are features used to build inference rules of the knowledge domain as well as to design the IE. The unification of facts is a basic task when designing algorithms for solving problems. The function of unification is also used in other procedures, such as checking the application of rules and checking the consistency of the KB. This manipulation plays an important role in the designing of each intellectual component and all the KBs. The process of the unification of facts involves the following steps:

- Determine the kind of facts.
- Check the equivalence of facts based on their kinds. The checking process belongs to the knowledge domain.
- In the inference process, the complexity of the unification of facts affects the complexity of this process. Thus the unification of facts is studied and solved effectively.

There are many traditional and modern methods for knowledge representation [35–37]. In fact, each method has strong points and weak points, and it is not really useful for application in the real knowledge domain. Thus designers determine a method or integrate it with other methods from the collected knowledge. Moreover, they also study the improvement and development of current methods to conform to practical knowledge domains. A full and accurate knowledge representation supports the designing of a KB that conforms to the requirements of the application, and a complete KB is also a good basis for designing an effective IE.

10.4.2 Design the Inference engine

10.4.2.1 The process for designing the inference engine

When designing a KBS, besides the organization of the KB, the IE is also very important. It plays as the reasoning process that simulates human thinking for searching for solutions to problems. This engine helps the system in solving problems in the represented knowledge domain.

10.4.2.1.1 The principles of an inference engine

The reasoning process of humans is a form of creative thinking, in which, from some knowledge, new knowledge is extracted. Thus the inference process must comply with certain principles [2,8,37]. Likewise, the IE of a KBS is designed to simulate this process, so this engine must be designed based on the following principles:

- *Logical reasoning*: The inference process of a KBS must satisfy the rules of logic. It also does not lead to a contradiction in the knowledge domain.
- *Consistent with represented knowledge*: The IE is designed based on the context of the specified knowledge, and the inference process must correspond to the KB of the system.

10.4.2.1.2 Criteria of an inference engine

There are two criteria for designing an IE:

- *The IE must be relatively independent of the KB*: Besides conforming to the specified KB, the inference process must be separate from the KB, that is, with the same goal for solving the same kind of problems, the deductive method of the system can be designed with many different KBs.
- *The inference process simulates human problem-solving*: The IE of a KBS shows the intelligence of the system. The method for problem-solving of the system has to simulate the human reasoning process. This makes the system more intelligent and highly interactive like a human.

10.4.2.1.3 The process for designing an inference engine

An IE is constructed based on the represented KB of the KBS for a certain knowledge domain. The process to achieve this design involves the following steps:

- **Step 1: Collect practical problems**

Before designing an IE, practical problems of the knowledge domain need to be collected. For example, with a diagnostic system, the medical records are collected in detail, and with an intelligent problem solver in education, the common exercises of the course need to be collected fully.

- **Step 2: Classify the collected problems and model them**

After collecting practical problems, they have to be classified. The classified kinds are the foundation to determining general problems of the knowledge domain. Following that, study the model for those general problems and design the corresponding algorithms. The models based on general problems are frames, or other models conforming to the KB.

- **Step 3: Design inference algorithms**

The algorithms for solving general problems are designed based on the combination of basic deductive strategies and heuristic rules. Those designing methods perform the intelligent problem-solving of the system. They use the KB as the core for the reasoning processes.

- **Step 4: Implement and test**

The next step is implementing the designed algorithms by using some programming tools, such as Maple, C#, and Python. After that, the program has to be tested on collected practical problems and other problems. Thereby, the evaluation of those algorithms is based on accuracy, complexity, ability to problem solve, etc.

- **Step 5: Increase the effectiveness of the inference engine**

The IE has to be updated regularly to increase its effectiveness. The improvement is performed by studying heuristic rules [6] or combining the A* algorithm [5] when searching for solutions. Those enhancement methods have to be used flexibly to best respond to the requirements of problem-solving.

10.4.2.2 The reasoning methods

The deductive methods are designed based on the reasoning process of humans. There are three kinds of human reasoning:

- *Deductive reasoning*: The process of reasoning from one or more statements to reach a logically certain conclusion based on the rules of the knowledge domain [5,6].
- *Inductive reasoning*: This reasoning method uses the premises of supplying some evidence but not fully ensuring the correctness of the conclusion [5,6].
- *Analogical reasoning*: The process of reasoning based on the idea that because two or more things are similar in some aspects, they are probably also similar in some further aspects [38].

Based on that reasoning, some inference processes for the KBS are proposed:

- Forward chaining: From the set of determined facts, inference rules of the knowledge domain are used to get new facts. This process is iterated until the goals of the corresponding problem are achieved.
- Backward chaining: From the goals of the problem, the inference rules are applied backward to arrive at the hypothesis of the current problem.
- Case-based reasoning: The method finds a solution by revision of the solutions of similar problems.
- Reasoning based on heuristic rules, knowledge of sample problems, and patterns: The inference process combines the use of heuristic rules. Besides, the study of patterns and sample problems of knowledge simulates the human reasoning process and gets the target facts quickly.

10.4.2.2.1 Forward chaining

Forward chaining simulates the method of deductive reasoning. The main idea behind this chaining: it begins with a set of determined facts and extracts new facts by applying rules which have their hypothesis matching known facts. This processing is iterated until the goals of the current problem are achieved.

The process of forward chaining is shown in the following diagram:

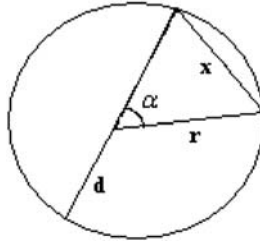
$$F_0 \xrightarrow{r_1} F_1 \xrightarrow{r_2} F_2 \xrightarrow{r_3} \dots \xrightarrow{r_m} F_m$$

in which r_i is an inference rule of the knowledge domain;

F_0 is the initial set of facts; it is the hypothesis of the problem;

$F_i = F_{i-1} \cup g(r_i)$, where $g(r_i)$ is the set of facts deduced from rule r_i . ($1 \leq i \leq m$).

Example 1: In the case of a knowledge domain pertaining to circles, there are some rules:



$$r_1: d = 2r \quad r_2: x = 2r \sin \frac{\alpha}{2}$$

$$r_3: P = 2\pi r \quad r_4: S = \pi r^2$$

where, d , r : diameter and radius of the circle, respectively; α : central angle of the circle; x : a chord of the circle; P , S : perimeter and area of a regular circle, respectively.

From the factors α and d , the inference process to compute area S and length of chord x is as follows:

$$F_0 \xrightarrow{r_1} F_1 \xrightarrow{r_2} F_2 \xrightarrow{r_4} F_3$$

with $F_0 = \{\alpha, d\}$ $F_1 = \{\alpha, d, r\}$ $F_2 = \{\alpha, d, r, x\}$ $F_3 = \{\alpha, d, r, x, S\}$.

10.4.2.2.2 Backward chaining

This method is performed by tracing the goal back to the hypothesis of the problem by applying rules in the KB. This backward inference will generate a tree with a rollback mechanism. The solution will be found when all the goals at the leaf nodes belong to the given facts. Backward chaining corresponds to a depth search on the graph representing the set of rules.

Compared to backward chaining, forward chaining is simpler. However, forward chaining has to use all inference rules without considering the rules that are relevant to the goals of the problem. Therefore this algorithm leads to a combinatorial explosion when the KB has a large number of inference rules. In contrast, backward chaining is more complex, but it only uses rules related to the goals.

10.4.2.2.3 Reasoning with pattern problems and sample problems

A *pattern problem* is a problem that has a simple solution and high frequency in the knowledge domain. It is a small problem, and its solution is not complex, but it usually appears when solving practical problems in the knowledge domain [39].

A *sample problem* is a high-frequency problem that is time consuming when using the general inference method to solve it. Its solution is recorded. When solving some practical problems similar to a sample problem, the sample problem is used to solve the current problem based on its recorded solution. If the current problem has little difference as compared to a sample problem, the solution of the current problem can be found more quickly [40].

Example 2: In the knowledge domain of plane geometry, there is a sample problem P: “In a circle with the center I and radius r , CD is a diameter and ND is a chord of the circle (I).”

The stored solution of P is:

$s_1: \{CD \text{ diameter Circle}(I)\} \rightarrow \{I \text{ midpoint } CD, C \text{ belong Circle}(I)\}$

$s_2: \{ND \text{ chord Circle}(I)\} \rightarrow \{N \text{ belong Circle}(I), D \text{ belong Circle}(I)\}$

$s_3: \{N \text{ belong Circle}(I), CD \text{ diameter Circle}(I)\} \rightarrow \{\Delta NCD: \text{Right Triangle}\}$

$s_4: \{\Delta NCD: \text{Right Triangle}, I \text{ midpoint } CD\} \rightarrow \{NI = CD/2\}$.

If the inference processing is posed with problem P, it will add four steps mentioned above into the processing. If the goals are not obtained, the process continues to deduce until the goals are achieved. The system also updates new solution steps into the problems.

Fig. 10.3 represents the reasoning method using pattern and sample problems that are related to the current problem for solving it [40,41]. This method simulates human reasoning, which has higher accuracy, for problem-solving. It gets results better than the other methods [7]. To solve an input problem, this reasoning uses sample and pattern problems to

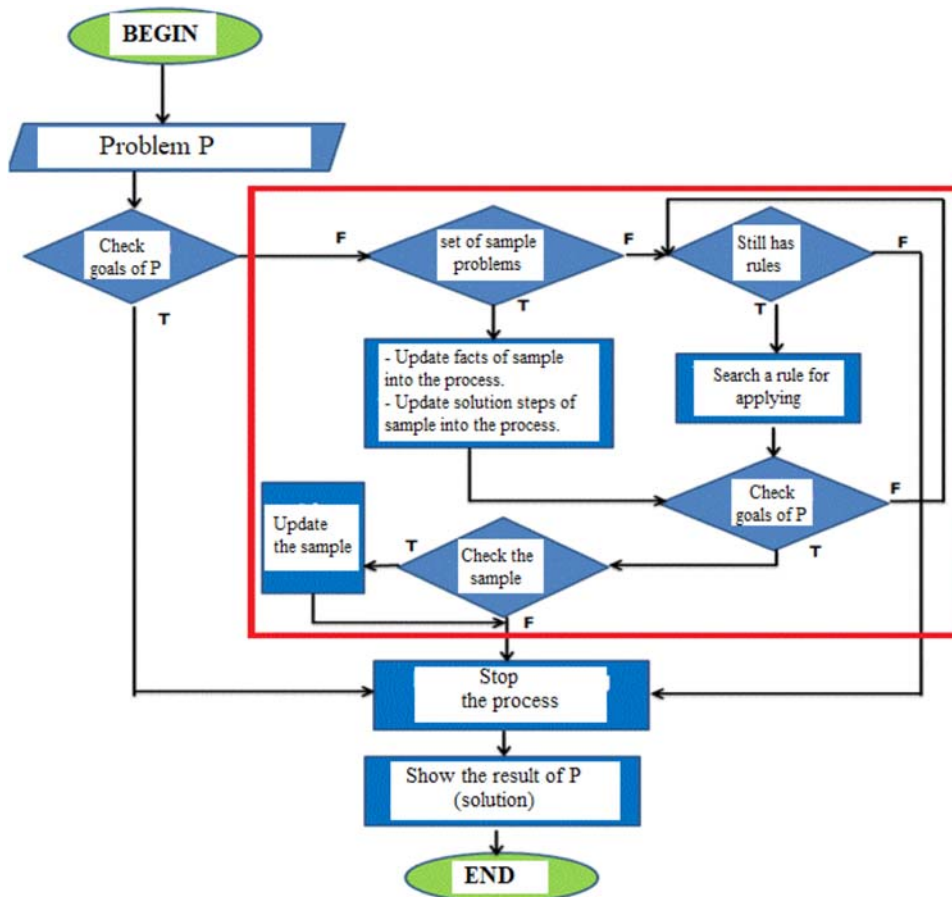


FIGURE 10.3 The reasoning that uses pattern and sample problems.

update the set of facts of the current problem. The sample problems are searched based on the unification of kinds of facts in the sample and facts in the current problem. After that, this reasoning uses the combination of forward and backward chaining by searching for inference rules that can be applied to the current set of facts. It also uses heuristic rules to select inference rules more effectively. The deductive process will be stopped if it achieves the goals to solve the problem or if it cannot get new facts in the reasoning.

10.5 Applications

10.5.1 Design an intelligent problem solver for solving solid geometry at high school

10.5.1.1 Collect the knowledge domain

The knowledge domain pertaining to high school solid geometry is collected from books [7]. The collected knowledge is classified as follows:

- *Conceptual knowledge*: In this application, some concepts of this domain are considered, such as point, segment, line, plane, triangle, quadrilateral, parallelogram, square, triangular pyramid, and quadrilateral pyramid. Each concept has a structure and behaviors in itself.
- *Relational knowledge between concepts*: Some relations in this knowledge domain are as Table 10.1.
- *Functional knowledge*: There are some functions for this knowledge domain as Table 10.2.
- *Rules of the knowledge domain*: Some of the collection rules of the knowledge domain are given as follows:

Rule 1: If a line is perpendicular to a plane, then that line is perpendicular to all lines on that plane.

$$\begin{cases} a, b: \text{Line} \\ a \perp \text{Plane}(P) \Rightarrow a \perp b \\ b \subset \text{Plane}(P) \end{cases}$$

TABLE 10.1 Relations between concepts of solid geometry.

Order	Relations	Properties
1	Point <i>belongs</i> Segment	
2	Point <i>belongs</i> Line	
3	Point <i>belongs</i> Plane	
4	Line <i>belongs</i> Plane	
5	Line <i>cuts</i> Line	Symmetric
6	Line <i>cuts</i> Plane	Symmetric
7	Two lines are <i>cross</i>	Symmetric
8	Three points are <i>straight</i>	Symmetric
9	Line <i>parallel</i> Line	Symmetric
10	Line <i>parallel</i> Plane	Symmetric
11	Plane <i>parallel</i> Plane	Symmetric
12	Line <i>perpendicular</i> Line	Symmetric
13	Line <i>perpendicular</i> Plane	Symmetric
14	Point is the <i>center</i> of Parallelogram	
15	Point is the <i>center</i> of Square	
16	Point is the <i>centroid</i> of Triangle	

TABLE 10.2 Some functions of solid geometry.

Order	Functions	Properties
1	Line IntersectionLine(P::Plane, Q::Plane)	Symmetric
2	Point IntersectionPoint(d::Line, P:: Plane)	Symmetric
3	Point IntersectionPoint(a:: Line, b:: Line)	Symmetric
4	Point Midpoint(A::Point, B::Point)	Symmetric
6	Point Projection(A:: Point, a:: Line)	
7	Point Projection(A:: Point, P:: Plane)	
8	Line Projection(d:: Line, P:: Plane)	
9	Line CommonPerpendicularLine(d:: Line, d1:: Line)	Symmetric
10	Real Distance(A:: Point, d:: Line)	Symmetric
11	Real Distance(A:: Point, P:: Plane)	Symmetric
12	Real Distance(d:: Line, d1:: Line)	Symmetric

Rule 2: If a line a does not belong to a plane (P) and it is also parallel to another line in (P), then line a is parallel to the plane (P).

$$\begin{cases} a, b: \text{Line} \\ a \not\subset \text{Plane}(P) \\ b \subset \text{Plane}(P) \\ a // b \end{cases} \Rightarrow a // \text{Plane}(P)$$

Rule 3: If a point belongs to two planes, then it belongs to the intersection line of those planes.

$$\begin{cases} K: \text{Point} \\ K \in \text{Plane}(P) \\ K \in \text{Plane}(Q) \end{cases} \Rightarrow K \in \text{Plane}(P) \cap \text{Plane}(Q)$$

- *Rules of the knowledge domain:* Some of the collection rules of the knowledge domain are given as follows:

Some of rules for generating new objects:

Rule 4: Properties of the intersection line between two planes.

$$\begin{cases} M \in \text{Plane}(P) \cap \text{Plane}(Q) \\ a \subset \text{Plane}(P) \\ b \subset \text{Plane}(Q) \\ a // b \end{cases} \Rightarrow \begin{cases} \text{Let } d = \text{IntersectLine}(P, Q): \\ d // a, d // b \end{cases}$$

Rule 5: In a triangle ABC, a point M belongs to AB, a point N belongs to BC. A new point P is generated with $P = CM \text{ intersect } AN$, which means P belongs to CM and P belongs to AN.

$$\begin{cases} \text{Triangle}(ABC) \\ M, N: \text{Point} \\ M \in AB \\ N \in BC \end{cases} \Rightarrow \begin{cases} \text{Let } P: \text{Point, and } P = \text{IntersectPoint}(CM, AN): \\ P \in CM, P \in AN \end{cases}$$

Some kinds of exercises from high school solid geometry in Ref. [7] are considered. Those kinds are:

- Prove a relation between two objects.
- Compute the value of a function related to objects.
 - Functions return a value;
 - Functions return an object.

10.5.1.2 Build the knowledge model

The knowledge model of solid geometry includes 5 components (**C**, **H**, **R**, **Funcs**, **Rules**) and 12 kinds of facts [3,32]. The details of each component for this knowledge model are shown as follows:

1. C—set of concepts

Set **C** is the set of concepts of the knowledge domain of solid geometry. Each concept has the structure of computational objects [7] and its behaviors for solving problems on itself. There are some types of computational objects for this knowledge domain:

- Basic object: Point.
- First-order objects: Segment, line, angle, plane.
- Second-order objects: Triangle and kinds of triangle, quadrilateral and its kinds.
- Third-order objects: Triangular pyramid, quadrilateral pyramid.

2. H—set of hierarchical relations:

The hierarchical relations between concepts in **C** can be represented by a Hasse diagram. Fig. 10.4 shows the Hasse diagram of hierarchical relations between the quadrilateral and its kinds.

3. R—set of relations between concepts

Set **R** is the set of binary relations between concepts in **C**. Those relations are not hierarchical relations.

Example 1: Relations *perpendicular* between two planes, two lines, or between a line and a plane.

Relations *belong* between a point and a line, or a point and a plane.

Relations *belong* between a point and a line, or a point and a plane. “Relation *the height* between a segment and a triangle, or a segment and a pyramid.”

4. Funcs—set of functions

Set **Funcs** is the set of functions on computational objects.

Example 2: The *IntersectionPoint* functions between two lines, or a line and a plane return a point.

The *IntersectionLine* function between two planes returns a line.

The *Projection* function between a line and a plane returns a line. And the *Projection* function between a point and a line returns a point.

Specification of a function:

function-def: = **FUNCTION** name;

ARGUMENT: argument-def +

RETURN: return-def;

[constraint]

[facts]

ENDFUNCTION;

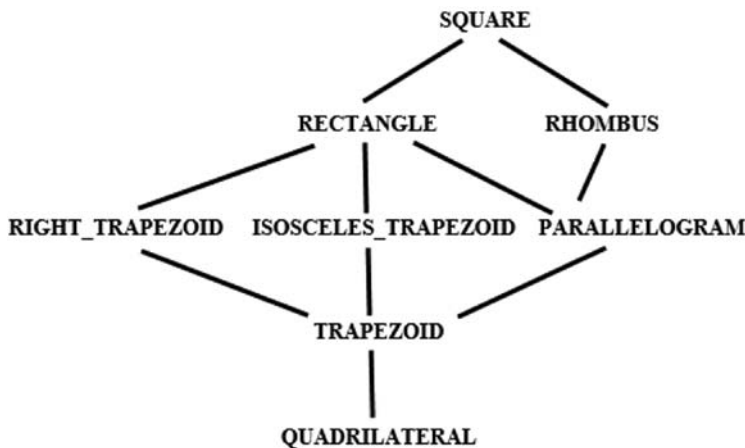


FIGURE 10.4 Hasse diagram for the quadrilateral and its kinds.

Example 3: The specification of the *IntersectPoint* function between a line and a plane:

```
function-def: = FUNCTION IntersectPoint;
ARGUMENT: d:Line, P: Plane
RETURN: M: Point;
Constraint: Not(d belong P)
IntersectionPoint(d, P)
M belong d.
M belong P
ENDFUNCTION;
```

5. Rules—set of inference rules

The set **Rules** is the set of deductive rules. Each rule has the form “if <facts> then <facts>.” There are four kinds of rules in solid geometry:

- Kind 1: *Natural rules*. The rules that can be deduced by a human without proving or explaining.
- Kind 2: *Rules for determining an object*. The rules that determine the existence of an object based on facts or other objects.
- Kind 3: *Rules for generating new objects*. The rules that generate new objects such as a point or line.
- Kind 4: Other rules.

Example 4 :

(Kind 2) Rule 1: Three points, which are not straight, determine a plane.

$\{A, B, C: \text{Point}; \text{not}(A, B, C \text{ straight})\} \rightarrow \{\text{Determine } \text{Plane}(ABC)\}$

(Kind 3) Rule 2: If two coplane lines intersect, there exists a point is the intersect point.

$$\left\{ \frac{d_1, d_2: \text{Line}; P: \text{Plane};}{d_1 \in P, d_2 \in P, d_1 \text{ intersect } d_2} \right\} \Rightarrow \{\exists M: \text{Point}, M = \text{IntersectPoint}(d_1, d_2)\}$$

(Kind 4) Rule 3: If a line has a point that does not belong to a plane, it does not belong to the plane.

$$\{d: \text{Line}; A: \text{Point}; P: \text{Plane}; A \notin P; A \in d\} \Rightarrow \{d \not\subset P\}$$

10.5.1.3 Organize the knowledge base

The KB of this knowledge domain is represented by the knowledge model as described in Section 10.1.2. It is organized by the system of structured text files with a specification language. Those text files are:

- The file *Object_Kinds.txt*, which stores the names of concepts of the knowledge domain;
- The files in the form “<Name_of_concept>.txt,” which store the structure of the corresponding concept;
- The file *Relations.txt*, which stores the relations between concepts;
- The file *Hierarchy.txt*, which stores the Hasse diagrams for the hierarchical relations between concepts;
- The file *Functions.txt*, which stores the definitions of functions in the knowledge domain;
- The file *Rules.txt*, which stores inference rules of the knowledge domain.

Example 5: The structure of the file *Quadrilateral_Pyramid.txt* describing the knowledge of a quadrilateral pyramid:

```
File Quadrilateral_Pyramid.txt
begin_concept: QuadrilateralPyramid[S,A,B,C,D];
S, A, B, C, D: Point
begin_variables
p_SAB: Plane[S, A, B];
p_SBC: Plane[S, B, C];
```

```

p_SDC: Plane[S, D, C];
p_SAD: Plane[S, A, D];
p_ABCD: Plane[A, B, C, D];
SA: Line[S, A];
SB: Line[S, B];
SC: Line[S, C];
SD: Line[S, D];
AB: Line[A, B];
BC: Line[B, C];
AC: Line[A, C];
CD: Line[C, D];
DA: Line[D, A];
SA_: Segment[S, A];
SB_: Segment[S, B];
SC_: Segment[S, C];
SD_: Segment[S, D];
AB_: Segment[A, B];
BC_: Segment[B, C];
AC_: Segment[A, C];
CD_: Segment[C, D];
DA_: Segment[D, A];
end_variables
begin_construction_properties
QuadrilateralPyramid[S, A, B, C, D] = QuadrilateralPyramid[S, B, C, D, A]
QuadrilateralPyramid [S, A, B, C, D] = QuadrilateralPyramid [S, C, D, A, B]
QuadrilateralPyramid [S, A, B, C, D] = QuadrilateralPyramid [S, D, A, B, C]
end_construction_properties
begin_properties
["belong", S, "Object"]
["belong", A, "Object"]
["belong", B, "Object"]
["belong", C, "Object"]
["belong", D, "Object"]
["co-plane", A, B, C, D]
["cross", SA, BC]
["cross", SA, CD]
["cross", SB, DA]
["cross", SB, CD]
["cross", SC, DA]
["cross", SC, AB]
["cross", SD, AB]
["cross", SD, BC]
Not["belong", S, p_ABCD]
Not["belong", A, p_SBC]
Not["belong", A, p_SDC]
Not["belong", B, p_SAD]
Not["belong", B, p_SDC]
Not["belong", C, p_SAD]
Not["belong", C, p_SAB]
Not["belong", D, p_SAB]
Not["belong", D, p_SBC]
end_properties
end_concept
Model of problems

```

The model of problems in the knowledge domain of high school solid geometry has the form:

$(\mathbf{O}, \mathbf{F}) \rightarrow \mathbf{G}$

where $\mathbf{O}$ is the set of objects of the problems, $\mathbf{F}$ is the set of facts, and $\mathbf{G}$ is the set of goals. Each goal is:

- Proving a relation between two objects.
- Determining the value of a function related to objects.
- Functions return a value;
- Functions return an object.

Example 6: Given a triangular pyramid S.ABC. Let M, N, and P be midpoints of segments AB, BC, and CD, respectively. Determine the intersection point Q between line AD and plane (MNP).

+ Model of this problem:

$$\begin{aligned} O: &= \left\{ \begin{array}{l} \text{Triangular Pyramid}[S, A, B, C], \\ M, N, P: \text{Point} \end{array} \right\} \\ F: &= \left\{ \begin{array}{l} M \text{ midpoint } AB, N \text{ midpoint } BC, \\ P \text{ midpoint } CD \end{array} \right\} \\ G: &= \{\text{Determine: IntersectPoint}(\text{line}(AD), \text{plane}(MNP))\} \end{aligned}$$

10.5.1.4 Design the inference engine

The algorithm for solving problems in this knowledge domain is designed based on a combination of forward and backward chainings, in which objects have the ability to reason themselves. The inference process also integrates heuristic rules to increase its effectiveness. Those heuristics rules speed up the process. Some of the heuristic rules are using sample problems that are similar to the corresponding problem [40] and rules for the limitation of generating new objects.

Fig. 10.5 illustrates the working of the algorithm for solving problems in the knowledge domain of solid geometry. At first, it collects the required information from the hypothesis of the problem, and then it initializes the solving method. When solving the current problem, the algorithm uses geometric objects as agents to solve problems by themselves for determining new facts. After that, the IE uses heuristic rules to select some rules which can be applied to the current status of the problem being solved. Using selected rules, the algorithm continues to deduce to get more new facts until the goals are reached or it does not have any rules that can be applied. Finally, the problem will be solvable if all its goals can be obtained, and otherwise, the problem will be unsolvable.

10.5.1.5 Testing

The intelligent problem solver to solve high school solid geometry problems has three components: user interface, KB, and module for solving problems. The user interface is implemented by using HTML and JavaScript. The module for solving problems is set up by using Maple, and C# is used to connect the user interface and the solving module. The KB is organized using the system of structured text files.

The user interface of this system is shown in Fig. 10.6

Example 7: The proof of Example 6 is found by using this program:

The proof of this system:

Step 1:{M midpoint AB} $\rightarrow$ {M belong AB} **Step 2:**{M belong AB, AB belong plane(ABD)} $\rightarrow$ {M belong plane(ABD)} **Step 3:**{M belong plane(ABD), M belong plane(MNP)} $\rightarrow$ {M belong IntersectionLine(plane(ABD), plane(MNP))} **Step 4:**{P midpoint CD} $\rightarrow$ {P belong CD} **Step 5:**{P belong CD, CD belong plane(ACD)} $\rightarrow$ {P belong plane(ACD)} **Step 6:**{A belong plane(ABD), A belong plane(ACD)} $\rightarrow$ {A belong IntersectionLine(plane(ABD), plane(ACD))} **Step 7:**{D belong plane(ABD), D belong plane(ACD)} $\rightarrow$ {D belong IntersectionLine(plane(ABD), plane(ACD))}

Step 14:{d1 = IntersectionLine(plane(ACD), plane(MNP)), AC belong plane(ACD), MN belong (MNP), AC // MN} $\rightarrow$ {d1 // AC, d1 // MN} **Step 15:**Let d2: line, d2 = IntersectionLine(plane(ABD), plane(MNP)) **Step 16:**{d2 = IntersectionLine(plane(ABD), plane(MNP))} $\rightarrow$ {d2 belong plane(ABD), d2 belong plane(MNP)} **Step 17:**{d2 = IntersectionLine(plane(ABD), plane(MNP))} $\rightarrow$ {M belong IntersectionLine(plane(ABD), plane(MNP))} $\rightarrow$ {M belong d2} **Step 18:**{M belong d2, M belong MN} $\rightarrow$ {M = IntersectionPoint(d2, MN)} **Step 19:**{M = IntersectionPoint(d2, MN)} $\rightarrow$ {d2 intersec MN}

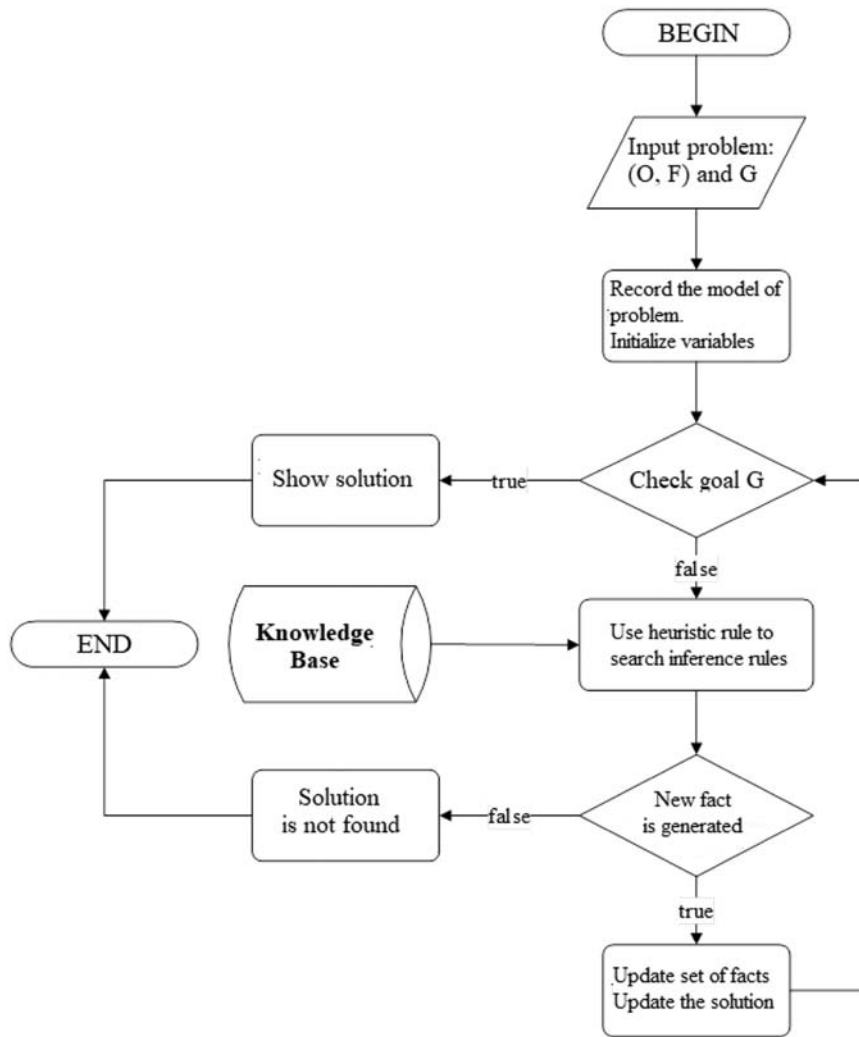


FIGURE 10.5 The algorithm for solving problems in the knowledge domain of solid geometry.

Step 8:{P belong plane(ACD), P belong plane(MNP)} → {P belong IntersectionLine(plane(ACD), plane(MNP))}
Step 9:{A belong IntersectionLine(plane(ABD), plane(ACD)), D belong IntersectionLine(plane(ABD), plane(ACD))} → {line(AD) = IntersectionLine(plane(ABD), plane(ACD))}
Step 10:{M midpoint AB, N midpoint BC} → {MN // AC}
Step 11:Let d1: line, d1 = IntersectionLine(plane(ACD), plane(MNP))
Step 12:{d1 = IntersectionLine(plane(ACD), plane(MNP))} → {d1 belong plane(ACD), d1 belong plane(MNP)}
Step 13:{d1 = IntersectionLine(plane(ACD), plane(MNP)), P belong IntersectionLine(plane(ACD), plane(MNP))} → {P belong d1}

Step 20:{d2 belong plane(MNP), MN belong (MNP), d1 belong (MNP), d1 // MN, d2 intersec MN} → {d1 intersec d2}
Step 21:{d1 intersec d2} → {H: point, H = IntersectionPoint(d1, d2)}
Step 22:{H = IntersectionPoint(d1, d2)} → {H belong d1, H belong d2}
Step 23:{H belong d1, d1 belong plane(ACD)} → {H belong plane(ACD)}
Step 24:{H belong d2, d2 belong plane(ABD)} → {H belong plane(ABD)}
Step 25:{H belong plane(ABD), H belong plane(ACD)} → {H belong IntersectionLine(plane(ABD), plane(ACD))}
Step 26:{H belong IntersectionLine(plane(ABD), plane(ACD))} → {H belong line(AD) = IntersectionLine(plane(ABD), plane(ACD))}
Step 27:{H belong d2, d2 belong plane(MNP)} → {H belong plane(MNP)}
Step 28:{H belong line(AD), H belong plane(MNP)} → {H = IntersectionPoint(line(AD), plane(MNP))}

10.5.2 Consultancy system for designing housing architecture

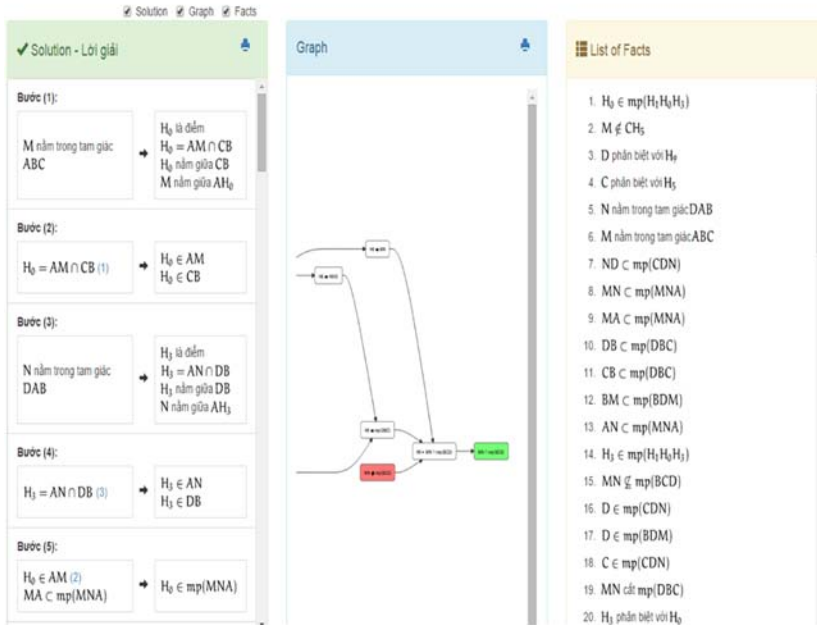
In Ref. [42], some solid geometry theorems are proved using the coordinate-free method based on volume. This method uses geometric variants between signed volume and area method to find solutions of theorems. Those solutions are readable and are represented step-by-step (Fig. 10.7).

Table 10.3 compares the ability of the proposed method and the coordinate-free method in Ref. [42] to organize the KB of solid geometry.

SOLID GEOMETRY SOLVING



(A)



(B)

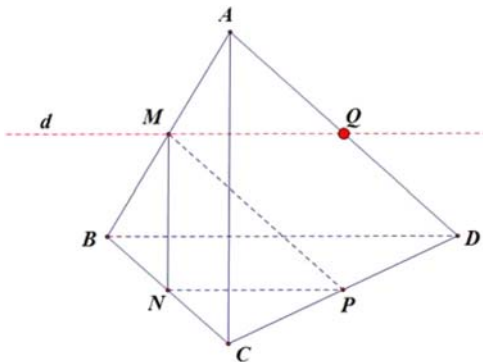


FIGURE 10.7 Representation of example 6.

TABLE 10.3 Organizing the knowledge base.

	Concepts	Relations	Rules
Quantity of knowledge in [7]	31	69	158
Knowledge base of the proposed method	27 concepts (87%) include:• 5 basic concepts: point, line, segment, plane, angle;• 13 concepts about objects in plane geometry: circle (1), triangle and its kinds (5), quadrangle and its kinds (7);• 9 concepts about polyhedrons in solid geometry: 1 for sphere, 4 kinds of pyramid, 4 kinds of prism;	61 relations (88%) include:• 33 relations between objects of plane geometry.• 28 relations between objects of solid geometry.	137 rules (87%) include:1. <i>Deductive rules</i> :• 51 rules of plane geometry;• 18 rules about parallel relation;• 29 rules about perpendicular relation;• 10 rules about angles.2. <i>Rules for generating a new object</i> :• 17 rules for generating a new point in parallel relation;• 8 rules for generating a new point in perpendicular relation.3. <i>Equivalent rules</i> : 4 rules.
Knowledge base of the coordinate-free method [42]	~13 concepts (41%) include:• 5 basic concepts: point, line, segment, plane, angle;• ~5 concepts about objects in plane geometry: triangle, quadrangle, circle.• ~3 concepts in solid geometry: tetrahedron, sphere.	~23 relations (33%) include:• Relations between objects of plane geometry;• Relations between objects of solid geometry, such as intersection, parallel perpendicular.	~70 rules (44%); it also includes some computing rules, such as Pythagorean differences of triangles, signed volumes of tetrahedrons, formulas to compute ratio, area, volume.

TABLE 10.4 The ability to solve exercises of the methods.

Exercises	Number of exercises	Number of solved exercises	
		Proposed method	Coordinate-free method
Kind 1	21	19	3
Kind 2	27	21	4
Kind 3	19	13	10
Kind 4	16	9	2
Total	83	62	19

There are 83 exercises collected from books [7] for testing. Those exercises are classified into the following kinds:

- Kind 1: Exercises about determining an intersection point of a line and a plane or an intersection line of two planes.
- Kind 2: Exercises about parallel relations between geometric objects.
- Kind 3: Exercises about perpendicular relations between geometric objects.
- Kind 4: General problems, which use properties of intersection and perpendicular and parallel relations for solutions.

Table 10.4 compares the ability of the proposed method and the coordinate-free method in Ref. [42] to solve exercises from these kinds of courses.

Both methods give solutions that are readable and represented step-by-step. The reasoning of the coordinate-free method uses geometric variants of the signed volume and area method; thus most of the exercises, which are transformed into area and volume, can be solved by using it. Hence, its proofs are still machinery and not suitable to help learners in studying. In contrast, the reasoning of the proposed method uses the knowledge of solid geometry to deduce new facts in its inference process directly. The method can solve exercises about perpendicular and parallel relations. Thus it can get the proofs that have reasoning similar to that of students, so it is useful for supporting the learning.

10.5.2.1 Organize the knowledge base of the consultancy system

In modern society, as the economy develops, the demand for housing is more diverse. Housing architecture is associated with material life, natural geography, and spirituality. Therefore for satisfying the factors of safety and aesthetics and harmonizing with the common space, nature space, and feng-shui, the housing architecture requires consultancy from and design of experts who are architects. Cheung et al. [43] proposed a set of common criteria for architectural projects. Besides, almost all present methods for building a project management consultant system are constructed through the analytical hierarchy process [44–46]. For example, the Architectural Consultant Selection System (ACSS) is a program evaluating prospective consultant architects and was constructed based on the analytical hierarchy process [43]. ACSS provides an efficient and consistent method for the selection of architects. However, it has not yet been established through building legislation, especially legal documents, and law application documents.

In this section, a consultancy system for designing housing architecture in Vietnam is proposed. This system is built based on the regulations of housing architecture in Ho Chi Minh City. It also has to meet the requirements to support the consultation of housing architecture: Firstly, the system has to be equipped with the local regulations of housing architecture in Vietnam. It must have the full information for consulting, such as the direction of the house, room spaces, balcony, garden, and stair style. Secondly, the system supports searching similar architectures, which need to be visualized for the investor.

10.5.2.2 Design the inference engine of the consultancy system

The regulations of housing architecture and construction are collected from Ref. [12]. They are applicable for designing city houses in Ho Chi Minh City (Vietnam). The KB of the consultancy system is organized by a knowledge model including five components, given as follows:

(C, Funcs, Rules, Pros, P)

1. C—Set of concepts

Each concept in C is a class that has the structure and is classified as follows:

- String variables: for example, the variable “*direction*” is a string.
- Basic objects: These objects have real-value and string attributes.

For example, the object “*Washing room*” includes attributes that are real and string values. This object is used to describe other objects “*Equipment of Washing room*.”

- First-order objects: These objects have attributes that are real values, string values, and basic objects.

For example, the object “*floor*” has attributes that are basic objects, such as “*washing room*” and “*living room*”.

- Second-order objects: These objects have attributes that are real values, string values, basic objects, and first-order objects.

For example, the object “*List of floors*” has attributes that are first-order objects, such as “*floor*.”

The structure of each object includes: (*Attributes, Conditions, Com-relation, InnerRules*)

- *Attributes*: Set of attributes which are real values, string values, basic objects, and other objects that have lower orders.
- *Conditions*: Set of conditions for each attribute.
- *Com-relation*: Set of computational relations on attributes. Based on these relations, an attribute can be determined from other attributes of the same objects.
- *InnerRules*: Set of deductive rules related to attributes of the corresponding objects.

2. Funcs—Set of functions on concepts in C

The functions are used for reasoning and computing an object. The kinds of functions are given in Table 10.5.

3. Rules—Set of inference rules of the knowledge domain

Each rule in Rules has the form: $\{p_1, p_2, \dots, p_n\} \Rightarrow \{q_1, q_2, \dots, q_m\}, \text{prior}$,

in which p_i and q_j are facts of the knowledge domain and *prior* is the priority level of the corresponding rule.

The kinds of facts are:

- Determining an object.
- Determining an attribute of an object.
- Determining an attribute with the conditions of its bound.
- Determining an object with the conditions related to another object.
- Changing an attribute of an object by applying a function.
- Calling a function to compute.

TABLE 10.5 The functions of the knowledge base.

Kind	Meaning	Form	Example
1	The function for basic object and real-value variable.	$\langle \text{Name of function} \rangle (\text{basic object, real})$	
2	The function for first-order objects.	$\langle \text{Name of function} \rangle (\text{first-order object})$	$\text{Area}(\text{floor})$: Compute the sum of room areas in the <i>floor</i> .
3	The function for first-order object and real-value variable.	$\langle \text{Name of function} \rangle (\text{first-order object, real})$	$\text{Decrease}(\text{floor}, x)$: Decrease the area of rooms in the <i>floor</i> by value x .
4	The function for basic object and first-order object.	$\langle \text{Name of function} \rangle (\text{basic object, first-order object})$	$\text{Insert}(\text{room}, \text{floor})$: Insert the <i>room</i> into the <i>floor</i> .
5	The function for first-order object and second-order object.	$\langle \text{Name of function} \rangle (\text{first-order object, second-order object})$	$\text{Add}(\text{room}, \text{List of floors})$: Add rooms into floors of the <i>List of floors</i> .

If all facts in the hypothesis of a rule are activated, the rule will be activated. The rule with a higher *prior* value will be used first. The values of *prior* in this knowledge domain are:

Highest priority: 15

Next priority: 10

Average priority: 5

Lowest priority: 0 (it also is the default value of *prior*).

The inner rules of an object have the highest priority; their values are 15. If rules must be first calculated in the process (those rules are presented in Ref. [12]), their values are 10. The rules describe the expert knowledge have priorities are 5.

4. Pros—Set of solutions

Each solution includes a perspective drawing and a construction drawing. The construction drawing has a list of objects: basic objects, first-order objects, and second-order objects. For example, the construction drawing of “Ground floor” has the object “living room” with the attributes: area = 25 m² and floor = 1, and the object “dining room” has the attributes: area = 20 m² and floor = 1. The perspective drawing has attributes that are code of solution and a list of objects.

5. P—Set of criteria for searching a solution.

Each criterion in set P has the structure: (*ObjSearch*, *ListAttr*, *Point*),

where *ObjSearch*: Information of the searching object;

ListAttr: List of attributes of the searching object and the variant for each attribute;

Point: The value of the corresponding criterion.

Design the inference engine of the consultancy system

Based on the KB of the consultancy system, there are two problems in the consultation of housing architecture:

Problem 1: Find the solution of the problem $H \rightarrow G$, in which:

H: The hypothesis of the problem. It is a set of determined attributes of an object.

G: The goal of the problem. It is a set of attributes that need to be computed.

Problem 2: From problem $H \rightarrow G$, search the designing method satisfying the results of G. This method includes: a perspective drawing and a construction drawing.

A. Algorithm for solving problem 1

Input: The set of H in the problem $H \rightarrow G$

Output: Compute values of attributes in G.

Algorithm 1 determines the values of attributes in goal G by applying rules in the KB. Firstly, the IE of the system searches the inner rules of objects that can be applied to the set of facts. Secondly, if goal G has not yet been

determined, the algorithm will continue to search inference rules of the knowledge domain. Those rules are represented by the regulations of housing architecture and construction.

Algorithm 1:

Step 0: Initialize

FactSet: = H; # set of facts

Found: = false;

flag: = true;

Sol: = {};

Step 1: if $G \subset \text{FactSet}$ then

Found: = true;

Goto Step

Else Goto Step 2.

Step 2: while flag and not(Found) do

flag: = false;

Apply rules in *InnerRules* of each object in FactSet to get a new fact.

If (a new fact is generated) then

flag: = true;

rforms: = set of rules in *InnerRules* were used;

news: = set of new facts which were generated by applying rForms on FactSet;

FactSet: = FactSet $\cup$ news;

Solution: = Sol $\cup$ {[rforms, news]};

If $G \subset \text{FactSet}$ then Found: = true;

Apply inference rules in **Rules** set to get a new fact.

If (a new fact is generated) then

flag: = true;

rforms: = set of inference rules in **Rules** were used;

news: = set of new facts which were generated by applying rforms on FactSet;

Solution: = Sol $\cup$ {[rforms, news]};

If $G \subset \text{FactSet}$ then Found: = true;

Step 3: If Found then

Sol is the solution to the problem;

Else The problem cannot be solved.

B. Algorithm for solving problem 2

Definition 1: Let two objects $obj_1, obj_2 \in \mathbf{C}$:

The attribute *attr* of object obj_1 is consistent with an attribute of object obj_2 if:

- (i) Objects obj_1 and obj_2 are of the same kind, and
- (ii) The name of *attr* is similar to the name of an attribute of object obj_2 .

Definition 2: Given an object $obj \in \mathbf{C}$, a solution $s \in \mathbf{Pros}$, and an object o in s :

An attribute *attr* of the object obj matches an attribute of object o if:

- (i) Attribute *attr* is consistent with an attribute of object o , and
- (ii) $|obj_2.attr - obj_1.attr| \leq \delta$, where δ is a constant of the knowledge domain.

Definition 3: Given two solutions $A, B \in \mathbf{Pros}$ and a set of facts F :

Solution A is better than solution B if the number of attributes matching A is more than the ones matching B.

Problem 2: Given the knowledge domain (**C**, **Funcs**, **Rules**, **Pros**, **P**) about housing architecture:

For solving the problem $H \rightarrow G$ in kind 1, the system searches the designing method to satisfy the results of G. This method includes: a perspective drawing and a construction drawing.

Input: The problem $H \rightarrow G$ as kind 1.

Output: Show k solutions in **Pros** which are the k -best solutions for the results of G .

From the requirements of the problem, algorithm 2 determines a set of criteria matching with those requirements through the values of attributes computed in Problem 1. After that, using sample solutions in **Pros**, the algorithm computes the matching point of each solution against the criteria and modifies the suitable solutions if needed. Finally, the algorithm ranks the list of solutions by matching points.

Algorithm 2:

Step 0: Initialize

$sumPoint$ is the set of members that is a tube $(p, point_p)$ in which $p \in \mathbf{Pros}$, and $point_p$ is the point of solution p when computing the matching point for each attribute in G and solution p .

$FactSet$: set of facts

Step 1: Compute the point for each solution in **Pros** based on the criteria in **P** and results in G .

For each p in **Pros**

$point_p := 0$

for each object obj in the solution p

if (all attributes of obj are consistent with some attributes of another object o in G) then

Search a criterion c_{obj} in **P** which has $c_{obj}.ObjSearch = obj$

if (all attributes of the object o match some attributes of the object obj) then

$point_p := point_p + c_{obj}.Point$;

$sumPoint := sumPoint \cup \{(p, point_p)\}$;

Step 2: Sort the solutions in $sumPoint$ by descending $point_p$.

Show k solutions in $sumPoint$.

10.5.2.3 Testing

The consultancy system for designing housing architecture is implemented using C#. The user interface of this program is given in Fig. 10.8.

Example 8: The example is obtained from Ref. [47].

The user inputs information about the house:

Information about the place:

Location: Ho Chi Minh city.

Area: countryside.

Kind of street: small street.

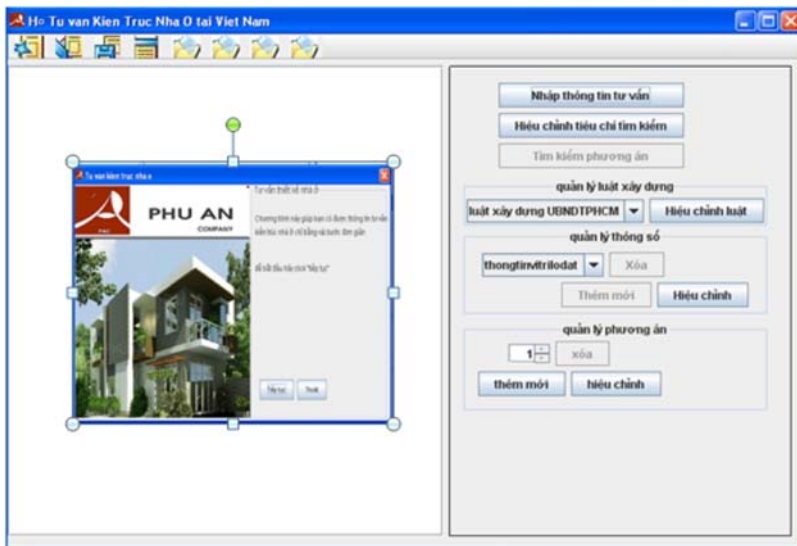


FIGURE 10.8 The user interface of the consultancy system for designing housing architecture.

Neighbors: center, mall.

Information about building line:

Line 1 = 12.0 m.

Line 2 = 0.0 m.

Line 3 = 0.0 m.

Return line = 1.

Information about the shape of ground:

Overall area = 40.0 m².

Constructing area = 40.0 m².

Width = 4.0 m.

Length = 10.0 m.

Style: city house.

Roof: flat.

Direction: south.

The program consults to the design of housing architecture in the following way:

Style: Based on the regulations of QĐ135–2007-UBNDTP-081207:

Maximum number of floors = 4.

Proposed number of floors = 2.

Setback space of top floor = 0 m.

Standard front elevation = 25.0 m.

The height of floors = 2 m.

Total ground area = 78.5 m².

Information about the first floor:

Minimum height of floor = 3.0 m.

Maximum height of floor = 7.0 m.

Proposed height of floor = 3.0 m.

Total ground area = 40.0 m².

Using area = 25.0 m².

Number of rooms = 5.

List of rooms: Rest room, kitchen, living room, working room, entertainment room.

Information about the second floor:

Minimum height of floor = 2.5 m.

Maximum height of floor = 3.5 m.

Proposed height of floor = 2.5 m.

Total ground area = 38.5 m².

Using area = 27.0 m².

Number of rooms = 3.

List of rooms: Rest room, kitchen, personal bedroom, large bedroom.

Number of clear floors = 1:

Minimum area = 1.5 m².

Reality area = 1.5 m².

Belong = second floor.

Number of large bedrooms = 1:

Minimum area = 15.0 m².

Reality area = 15.0 m².

Belong: second floor.

Direction: east.

Number of restrooms = 2:

Minimum area = 2.0 m².

Reality area = 2.0 m².

Belong: first floor, second floor.

Direction: west.

Living room:

Minimum area = 6.0 m².

Reality area = 7.0 m^2 .

Belong: first floor.

Direction: east.

Should be near the kitchen and working room.

Kitchen:

Minimum area = 3.0 m^2 .

Reality area = 7.0 m^2 .

Belong: first floor.

Direction: west.

Should be near the living room.

Working room:

Minimum area = 5.0 m^2 .

Reality area = 5.0 m^2 .

Belong: first floor.

Direction: east.

If it is outside work, the room should be near the living room.

If it is private work, the room should be near the bedroom.

Corridor:

Minimum area = 1.0 m^2 .

Belong: second floor.

Direction: west.

Terrace:

Minimum area = 7.5 m^2 .

Belong: second floor.

Position: front.

Garden:

Minimum area = 0 m^2 .

Belong: first floor.

Direction: west.

Stairs:

Type: U-shaped stairs.

Minimum area = 4.0 m^2 .

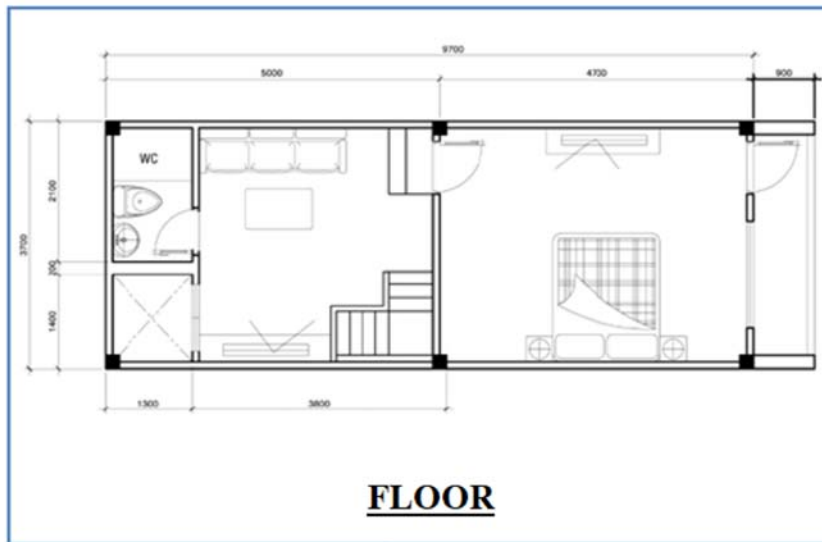
Reality area = 4.0 m^2 .



FIGURE 10.9 The perspective drawing of Example 8 [47].



(A)



(B)

FIGURE 10.10 (A) The construction drawing of the ground floor. (B) The construction drawing of the first and second floors.

The perspective drawing of this architecture is given in [Fig. 10.9](#).
The construction drawing of this architecture is given in [Fig. 10.10](#).

10.6 Conclusion and Future work

In this chapter, the architecture and the method for designing KBSs are proposed. Besides, the processes for constructing the components of those systems are also constructed, especially the KB and the IE. Both the KB and IE, which are built by using the proposed method, can meet the requirements to be applied in real-world intelligent systems.

The KB is designed based on gathering information from the knowledge domain. The knowledge has main components, namely concepts, relations, and inference rules. The collected knowledge is represented by using the methods of knowledge representation. Besides, the KB also has some basic knowledge manipulations, such as knowledge-updating, checking the knowledge's consistency, and unifications of facts. The IE is constructed using the problems of the

knowledge domain. Its reasoning process combines reasoning methods and heuristic rules to get the most effective deduction. The designing of KBs and IEs are established bases on the criteria of intelligent systems.

The proposed method for designing KBSs is useful for designing practical intelligent systems. It has been applied to build an intelligent problem solver for high school solid geometry problems. The method can represent the knowledge domains therein completely and solve some problems on them. It can represent more than 80% quantity of the knowledge in the selected course and solve 62 out of 83 tested exercises with step-by-step solutions and the reasoning for the answers using appropriate, high-school-level knowledge. Moreover, the built system meets the requirements of an intelligent system that supports learning, and it is emerging to be applied in the real world. Besides, the proposed method is applied to build a consultancy system for designing housing architecture based on the Vietnamese regulations of architecture and construction. This consultancy system has a KB including almost all rules in Ref. [12]. It can consult on some common housing architectures based on the requirements input by the user. Those architectures match with the government regulations.

In the future, some methods for knowledge management will be studied deeper [48]. They will be able to process the completeness and consistency of a KB in an intelligent system when it is updated. Nowadays, a real-world knowledge domain is integrated from many sources [49,50]. Thus a method for integrating many knowledge domains would be studied to organize the KB closer to reality [15,51,52]. Especially, the designed method will be applied to build some practical applications in other real-world domains, such as recommendation systems in healthcare, public administration services, intelligent searching on knowledge for building intelligent searching chatbots [53].

References

- [1] R. Akerkar, P. Sajja, *Knowledge-Based Systems*, Jones & Bartlett Learning, 2010.
- [2] F. Harmelen, V. Lifschitz, B. Porter, *Handbook of Knowledge Representation*, Elsevier, 2008.
- [3] N.V. Do, Intelligent problem solvers in education: design method and applications, in: V.M. Koleshko (Ed.), *Intelligent Systems*, Intech, 2012, pp. 121–148.
- [4] M. Chein, M. Mugnier, *Graph-Based Knowledge Representation—Computational Foundations of Conceptual Graphs*, Springer-Verlag, London, 2009.
- [5] W. Ertel, *Introduction to Artificial Intelligence*, Springer, 2011.
- [6] S. Russell, P. Norvig, *Artificial Intelligence—A Modern Approach* (, third ed., Pearson, 2016.
- [7] Vietnam Ministry of Education and Training, *Textbook and Workbook of Geometry in High School*, Publisher of Education, 2019.
- [8] J.F. Sowa, *Knowledge Representation: Logical, Philosophical and Computational Foundations*, Brooks/Cole, 2000.
- [9] S.P. Kumar, Knowledge-based expert system in manufacturing planning: state-of-the-art review, *International Journal of Production Research* 57 (15–16) (2019) 4766–4790.
- [10] H. Yoshizumi, K. Hori, K. Aihara, The dynamic construction of knowledge-based systems, in: C.T. Leondes (Ed.), *Knowledge-Based System*, 2, Elsevier, 2010.
- [11] H. Nguyen, D. Nguyen, V. Pham, Design an intelligent problems solver about solid geometry based on knowledge model about relation, in: *Proceedings of 2016 IEEE International Conference on Knowledge and Systems Engineering (KSE 2016)*, Ha Noi, Vietnam, October 2016, pp. 150–155.
- [12] People Committee of Ho Chi Minh City, The regulations of the housing architecture in Ho Chi Minh city, Number of regulation 135/2007/UBNDTP on 8 December, 2007.
- [13] H.D. Nguyen, N.V. Do, N.P. Tran, X.H. Pham, V.T. Pham, Some criteria of the knowledge representation method for an intelligent problem solver in STEM education, *Applied Computational Intelligence and Soft Computing* 2020 (2020). Article ID 9834218.
- [14] R. Plant, R. Gamble, Methodologies for the development of knowledge-based systems, 1982–2002, *The Knowledge Engineering Review* 18 (01) (2003) 47–81.
- [15] H.D. Nguyen, N.V. Do, V.T. Pham, A. Selamat, E. Herrera-Viedma, A method for knowledge representation to design intelligent problems solver in mathematics based on Rela-Ops model, *IEEE Access* 8 (2020) 76991–77012.
- [16] A. Gunter, C. Kuhn, *Knowledge-Based Configuration—Survey and Future Directions*, in: F. Puppe (Ed.), *XPS-99*, LNAI 1570, Springer, 1999, pp. 47–66.
- [17] Y. Fu, J. Zhuang, Y. Chen, L. Guo, Y. Wang, A framework for optimizing extended belief rule base systems with improved Ball trees, *Knowledge-Based System* (2020) Retrieved from. Available from: <https://doi.org/10.1016/j.knsys.2020.106484>.
- [18] C. Ramirez, B. Valdes, A general knowledge representation model of concepts, in: C. Ramirez (Ed.), *Advances in Knowledge Representation*, InTech, 2012, pp. 43–76.
- [19] Y. Wang, O.A. Zatarain, A novel machine learning algorithm for cognitive concept elicitation by cognitive robots, in: I. Management Association (Ed.), *Cognitive Analytics: Concepts, Methodologies, Tools, and Applications*, IGI Global, 2020, pp. 638–654.
- [20] A. Felfernig, R. Burke, Constraint-based recommender systems: technologies and research issues, in: *Proceedings of 10th International Conference on Electronic Commerce (ICEC 2008)*, Innsbruck, Austria, 2008, pp. 1–10.
- [21] L.V. Naykhanova, I.V. Naykhanova, Conceptual model of knowledge base system, *Journal of Physics: Conference Series* 1015 (2018) 032097.

- [22] W. Yang, C. Fu, X. Yan, Z. Chen, A knowledge-based system for quality analysis in model-based design, *Journal of Intelligent Manufacturing* 31 (2020) 1579–1606 Retrieved from. Available from: <https://doi.org/10.1007/s10845-020-01535-8>.
- [23] C. Tong, D. Sriram, *Artificial Intelligence in Engineering Design—3*, Elsevier, 1992.
- [24] D. Oragui, Tacit knowledge: definition, examples, and importance. Retrieved from <<https://helpjuice.com/blog/tacit-knowledge>>, 2020.
- [25] J. Bobadilla, F. Ortega, A. Hernando, A. Gutiérrez, Recommender systems survey, *Knowledge-Based Systems* 46 (2013) 109–132.
- [26] G. Adomavicius, A. Tuzhilin, Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions, *IEEE Transactions on Knowledge and Data Engineering* 17 (6) (2005) 734–749.
- [27] Y. Koren, R. Bell, C.H. Volinsky, Matrix factorization techniques for recommender systems, *IEEE Computer* 42 (8) (2009) 42–49.
- [28] S. Zhang, Cost-sensitive KNN classification, *Neurocomputing* 391 (2020) 234–242.
- [29] P. Dai, Y. Yang, M. Wang, R. Yan, Combination of DNN and improved KNN for indoor location fingerprinting, *Wireless Communications and Mobile Computing* 2019 (2019) Article ID 4283857. Retrieval from. Available from: <https://doi.org/10.1155/2019/4283857>.
- [30] N.V. Do, Ontology COKB for knowledge representation and reasoning in designing knowledge-based systems, *Communications in Computer and Information Science (CCIS)* 513 (2015) 101–118.
- [31] Y. Wang, et al., Brain-inspired systems: a transdisciplinary exploration on cognitive cybernetics, *Humanity and Systems Science Towards Autonomous AI. IEEE SMC Magazine* 6 (1) (2020) 6–13.
- [32] N. Do, H. Nguyen, A. Selamat, Knowledge-based model of expert systems using rela-model, *International Journal of Software Engineering and Knowledge Engineering* 28 (8) (2018) 1047–1090.
- [33] P. Sajja, R. Akerka, Knowledge-based systems for development, in: A. Sajja (Ed.), *Advanced Knowledge Based Systems: Model, Applications & Research*, TMRF e-Book, 2010, pp. 1–11.
- [34] N. Do, H.D. Nguyen, A knowledge model about relations and application, in: *Proceedings of 4th International Conference on Data Mining and Intelligent Information Technology Applications (ICMIA 2012)*, Taiwan, 2012, pp. 701–704.
- [35] G. Jakus, V. Milutinovic, S. Omerovic, S. Tomazic, *Concepts, Ontologies, and Knowledge Representation*, Springer, 2013.
- [36] M. Berman, *A Knowledge Representation Practionary*, Springer, 2018.
- [37] R. Brachman, H. Levesque, *Knowledge Representation and Reasoning*, Morgan Kaufmann, Elsevier, 2004.
- [38] W. Kupers, Analogical reasoning, in: N.M. Seel (Ed.), *Encyclopedia of the Sciences of Learning*, Springer, Boston, MA., 2012.
- [39] N. Do, H. Nguyen, A reasoning method on computational network and its applications, in: *Proceedings of 2011 International MultiConference of Engineers and Computer Scientists (IMECS 2011)*, Hongkong, 2011a, pp. 137–141.
- [40] N.V. Do, H.D. Nguyen, T.T. Mai, Designing an intelligent problems solving system based on knowledge about sample problems, in: *Proceedings of 5th Asian Conference on Intelligent Information and Database Systems (ACIIDS 2013)*, Kuala Lumpur, Malaysia, LNAI 7802, Springer, 2013, pp. 465–475.
- [41] N. Do, H. Nguyen, A reasoning method on knowledge base of computational objects and designing a system for automatically solving plane geometry problems, in: *Proceedings of World Congress on Engineering and Computer Science 2011 (WCECS 2011)*, San Francisco, 2011b, pp. 294–299.
- [42] J. Jiang, J. Zhang, A review and prospect of readable machine proofs for geometry theorems, *Journal of System Science and Complexity* 25 (2012) (2012) 802–820.
- [43] F. Cheung, J. Kuen, M. Skitmore, Multi-criteria evaluation model for the selection of architectural consultants, *Construction Management and Economics* 20 (7) (2002) 569–580.
- [44] A. Jabbarzadeh, Application of the AHP and TOPSIS in project management, *Journal of Project Management* 3 (2) (2018) 125–130.
- [45] H. Pham, N. Do, H. Nguyen, A consulting system for estimating costs of an information technology hardware project based on law of public investment, in: *Proceedings of 7th IEEE International Conference on System Science and Engineering (ICSSE 2019)*, Quang Binh, Vietnam, 2019, pp. 285–290.
- [46] H. Tran, L. Le, Y. Lee, A fuzzy AHP model for selection of consultant contractor in bidding phase in Vietnam, *Journal of Construction Engineering and Project Management* 5 (2) (2015) 35–43.
- [47] N. Do, H. Nguyen, *Knowledge-Based Systems*. Publisher of Vietnam National University, Ho Chi Minh city, 2017 (Vietnamese).
- [48] F. Correa, D. Carvalho, Holistic knowledge management: adherence analysis of the Castillo and Cazarini model, *Knowledge Management Research & Practice* 18 (4) (2019) 439–449.
- [49] M. Dumontier, C.J. Baker, J. Baran, et al., The SemanticScience Integrated Ontology (SIO) for biomedical research and knowledge discovery, *Journal of Biomedical Semantics* 5 (2014) 14.
- [50] R. Salakhutdinov, Integrating domain-knowledge into deep learning, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Anchorage, Alaska, 2019.
- [51] N. Do, H. Nguyen, T. Mai, A method of ontology integration for designing intelligent problem solvers, *Applied Science* 9 (18) (2019) 3793.
- [52] S. Fratini, T. Nogueira, N. Policella, Integrating modeling and knowledge representation for combined task, resource and path planning in robotics, in: *Proceedings of the Workshop on Knowledge Engineering for Planning and Scheduling—27th International Conference on Automated Planning and Scheduling (ICAPS 2017)*, Pittsburgh, PA, 2017.
- [53] H. Nguyen, V. Pham, D. Tran, T. Le, Intelligent tutoring chatbot for solving mathematical problems in high-school, in: *Proceedings of 2019 11th IEEE International Conference on Knowledge and Systems Engineering (KSE 2019)*, Da Nang, Vietnam, Oct., 2019, pp.145–150.